# Основы компиляции

# Основы работы с командной строкой Linux

```
sab@SAB:.../Lesson1$
```

nano Program.c

```c
#include <stdio.h>
int main()
{
        printf("Hello World!\n");
        return 0;
}
```

**gcc Program.c –o Program**

```
sab@SAB:.../Lesson1$ ./Program
Hello World!
```

Task 001

# Решим простую задачу

Написать процедуры для сложения и перемножения чисел
Вывести наименьший результат

# Решим простую задачу

```
#include <stdio.h>
#define X 5

int summ(int x, int y) //comment
{
    return x + y;
}

int mult(int x, int y)
{
    return x * y;
}

int main()
{
    printf("Sum = %d\n",    summ(X,5));
    printf("Mult = %d\n",   mult(5,5));
    return 0;
}
```

Можно ли сделать лучше?

Task 005

# Разбиение программы на модули

lib.h

```c
int summ(int, int);
int mult(int, int);
```

lib.c

```c
int summ(int x, int y)
{
    return x + y;
}

int mult(int x, int y) //comment
{
    return x * y;
}
```

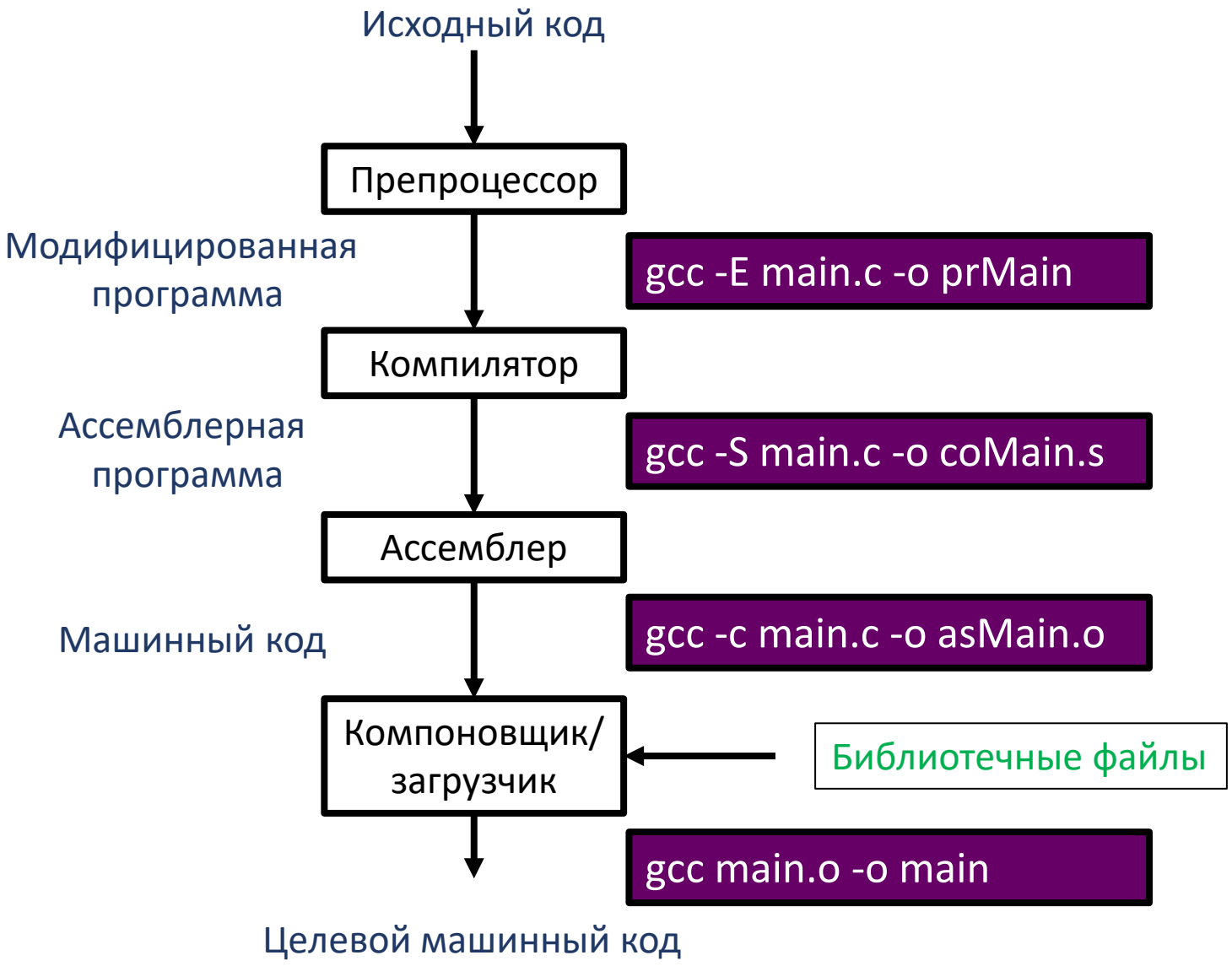
main.c

```c
#include <stdio.h>
#include "lib.h"

#define X 5

int main()
{
    printf("Sum = %d\n", summ(X,5));
    printf("Mult = %d\n", mult(5,5));
    return 0;
}
```

Как правильно скомпилировать?

Task 002

# Этапы компиляции



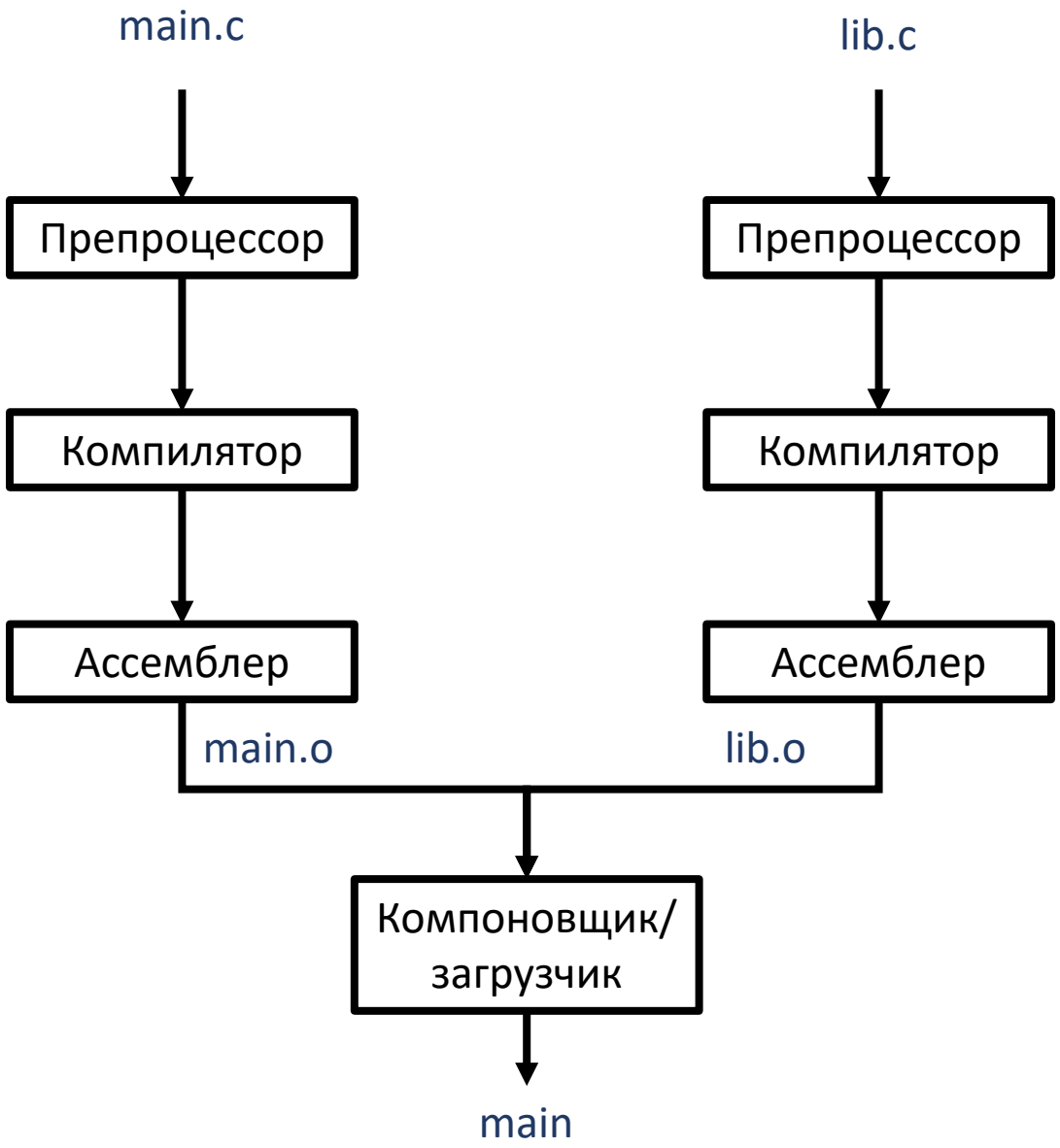

Task 005

# Кратко об ассемблере

| Регистр | Назначение |
|---|---|
| %eax | хранение результатов промежуточных вычислений |
| %ebx | хранения адреса (указателя) на некоторый объект в памяти |
| %ecx | счетчик |
| %edx | хранения результатов промежуточных вычислений |
| %esp | содержит адрес вершины стека |
| %ebp | указатель базы кадра стека |
| %esi | индекс источника |
| %edi | индекс приёмника |

| Команда | Назначение |
|---|---|
| **mov** *источник, назначение* | копирование *источника* в *назначение* |
| **lea** *источник, назначение* | помещает адрес *источника* в *назначение* |
| **add** *источник*, приёмник | *приёмник* = *приёмник* + *источник* |
| **sub** *источник*, приёмник | *приёмник* = *приёмник* - *источник* |
| **push** *источник* | поместить в стек |
| **pop** *назначение* | извлечь из стека |
| **cmp** *операнд_2, операнд_1* | *операнд_1 – операнд_2* и устанавливает флаги |
| **jle** *метка* | Переход если <= |

# Этапы компиляции



# Разбиение программы на модули

lib.h

```c
int summ(int, int);
int mult(int, int);
```

lib.c

```c
int summ(int x, int y)
{
    return x + y;
}

int mult(int x, int y) //comment
{
    return x * y;
}
```

main.c

```c
#include <stdio.h>
#include "lib.h"

#define X 5

int main()
{
    printf("Sum = %d\n", summ(X,5));
    printf("Mult = %d\n", mult(5,5));
    return 0;
}
```

Как правильно скомпилировать?

```
gcc main.c -c -Werror -Wall -g -o main.o
gcc lib.c -c -o lib.o
gcc main.o lib.o -o main
./main
```

# Разбиение программы на модули

lib.h

```c
int summ(int, int);
int mult(int, int);
```

lib.c

```c
int summ(int x, int y)
{
    return x + y;
}

int mult(int x, int y) //comment
{
    return x * y;
}
```

main.c

```c
#include <stdio.h>
#include "lib.h"

#define X 5

int main()
{
    printf("Sum = %d\n", summ(X,5));
    printf("Mult = %d\n", mult(5,5));
    return 0;
}
```

Как правильно скомпилировать?

```
gcc main.c -c -Werror -Wall -g -o main.o
gcc lib.c -c -o lib.o
gcc main.o lib.o -o main
./main
```

Что будет, если подключить заголовочный файл несколько раз?

# Директивы препроцессора

## Как избежать ошибки с многократным включением .h файла?

```
#define X
#ifdef X
/* Если до этого символ X был определён, то
включить текст до #endif. */
#endif

#ifndef X
/* Если до этого символ X НЕ был определён, то
включить текст до #endif. */
#endif
```

```c
#include <stdio.h>
#define X
int main()
{
    int a = 0;
#ifdef X
    a = 10;
#endif

#ifndef X
    a = 20;
#endif

    printf("%d\n", a);
    return 0;
}
```

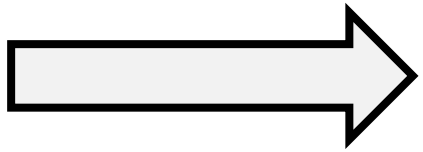
gcc -E prog.c -o prog.ii

Task 003

# Директивы препроцессора

## Как избежать ошибки с многократным включением .h файла?

```
void summ(int, int, int *);
void mult(int, int, int *);
```



```
#ifndef FILE_H
#define FILE_H
void summ(int, int, int *);
void mult(int, int, int *);
#endif
```

# Флаги компиляции

```c
#include <stdio.h>
#include <math.h>

int main() {
    double num;
    printf("Введите число: ");
    scanf("%lf", &num);
    printf("\n=== Результаты вычислений ===\n");
    printf("Квадратный корень: %.3f\n", sqrt(num));
    printf("Синус: %.3f\n", sin(num));
    printf("Косинус: %.3f\n", cos(num));
    printf("Натуральный логарифм: %.3f\n", log(num));
    printf("Экспонента: %.3f\n", exp(num));
    printf("Модуль: %.3f\n", fabs(num));

    return 0;
}
```

```
gcc main.c -lm -o main
```

Task 006

# Флаги компиляции

```c
#include <stdio.h>


struct Empty {
};


int main() {
    int x;
    int y = 10;
    printf("Size: %zu\n", sizeof(struct Empty));
    return 0;
}
```

```
gcc -Wall main.c -o main
gcc -Wpedantic main.c -o main
gcc -Wall -Wpedantic -Werror main.c -o main
```

Task 007

# Форматирование кода

```c
#include <stdio.h>
#include <stdlib.h>

int main(){
      int x = 5;
int arr[100];
      arr[1000] = 20;
printf("Hello");
if (x > 3)
{
printf("World");
}
return 0;
}
```

```
clang-format -i main.c
clang-tidy main.c -checks=*
```

Task 008

# Сборка программ

# Make файлы

цель: зависимости

**[tab]** команда

```
gcc main.c -c -o main.o
gcc lib.c -c -o lib.o
gcc main.o lib.o -o main
```

Makefile

```
all: clean
        gcc -c *.c
        gcc *.o -o main
clean:
        rm -f *.o
```

```
CFLAGS=-Wall -g -Werror
all: clean
        gcc –c $(CFLAGS) *.c
        gcc *.o -o main
clean:
        rm -f *.o
```

**Task 012**

# Разбиение программы на модули

lib.h

```c
#pragma once
int summ(int, int);
int mult(int, int);
```

lib.c

```c
int summ(int x, int y)
{
    return x + y;
}

int mult(int x, int y) //comment
{
    return x * y;
}
```

main.c

```c
#include <stdio.h>
#include "lib.h"

#define X 5

int main()
{
    printf("Sum = %d\n", summ(X,5));
    printf("Mult = %d\n", mult(5,5));
    return 0;
}
```

```
gcc main.c -c -Werror -Wall -g -o main.o
gcc lib.c -c -o lib.o
gcc main.o lib.o -o main
./main
```

# Make файлы

цель: зависимости
[tab] команда

Makefile

```
all: clean
        gcc -c *.c
        gcc *.o -o main
clean:
        rm -f *.o
```

```
gcc main.c -c -o main.o
gcc lib.c -c -o lib.o
gcc main.o lib.o -o main
```

```
CFLAGS=-Wall -g -Werror
all: clean
        gcc -c $(CFLAGS) *.c
        gcc *.o -o main
clean:
        rm -f *.o
```

```
make
```

# .PHONY

Создадим в директории с Makefile файл *clean*

```
CFLAGS=-Wall -g -Werror
all: clean
        gcc -c $(CFLAGS) *.c
        gcc *.o -o main
clean:
        rm -f *.o
```

```
$ make clean
make: 'clean' is up to date.
```

```
.PHONY: clean all
CFLAGS=-Wall -g -Werror
all: clean
        gcc -c $(CFLAGS) *.c
        gcc *.o -o main
clean:
        rm -f *.o
```

```
$ make clean
make: 'clean' is up to date.
```

# Еще чуть усложним

```
.PHONY: clean all
CC = gcc
CFLAGS = -Wall -Wpedantic -Werror
TARGET = main
OBJECTS = main.o

$(TARGET): $(OBJECTS)
    $(CC) $(OBJECTS) -o $(TARGET)

%.o: %.c
    $(CC) -c $(CFLAGS) $< -o $@

clean:
    rm -f $(OBJECTS) $(TARGET)
```

| Переменная | Значение |
|---|---|
| $@ | имя цели |
| $^ | все зависимости |
| $< | первая зависимость |
| $? | изменённые зависимости |
| $* | имя без расширения |

```
OBJECTS = $(SOURCES:.c=.o)
```

Task 013

# cmake

**CMake** (cross-platform make) — кроссплатформенное программное средство автоматизации сборки программного обеспечения из исходного кода. **Не занимается непосредственно сборкой**, а лишь **генерирует файлы сборки** из предварительно написанного файла сценария CMakeLists.txt и предоставляет простой единый интерфейс управления.

*CMakeLists.txt*

```
cmake_minimum_required(VERSION 3.16)
project(main)
set(SOURCE main.c)
add_executable(main ${SOURCE_EXE})
```

cmake *CMakeLists.txt*
make

cmake --build .

Task 015

# Задача на подумать

- Напишите программу на Си, которая содержит две функции:
  - Печать ФИО на экран
  - Печать номера группы на экран
- Разбейте программу на отдельные файлы – main.c, user.c, group.c
- Обязательно добавьте заголовочные файлы
- Соберите программу, используя Makefile
- Запустите программу и продемонстрируйте результат преподавателю